

Chapter 8: Memory- Management Strategies

Chien Chin Chen

Department of Information Management
National Taiwan University

Outline

- Background
- Contiguous Memory Allocation
- Paging
- Structure of the Page Table
- Segmentation

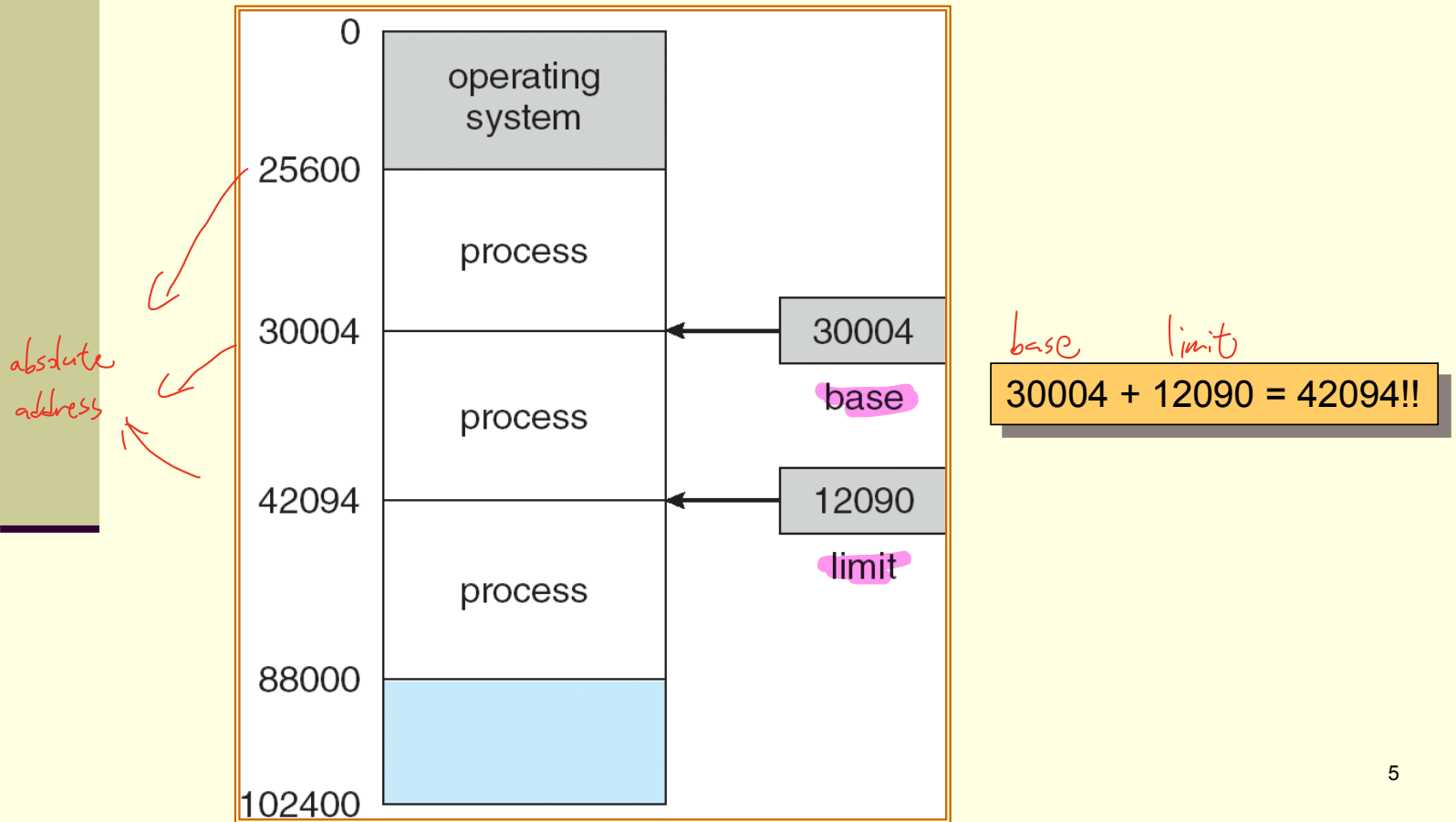
Background (1/5)

- Memory consists of a large array of words or bytes, each with its own address.
- It is central to the operation of a modern computer system.
 - In a instruction-execution cycle:
 - The CPU fetches instructions from **memory** according to the value of the program counter.
 - The instruction is then decoded and may cause operands to be fetch from **memory**.
 - Then, results may be stored back in **memory**.

Background (2/5)

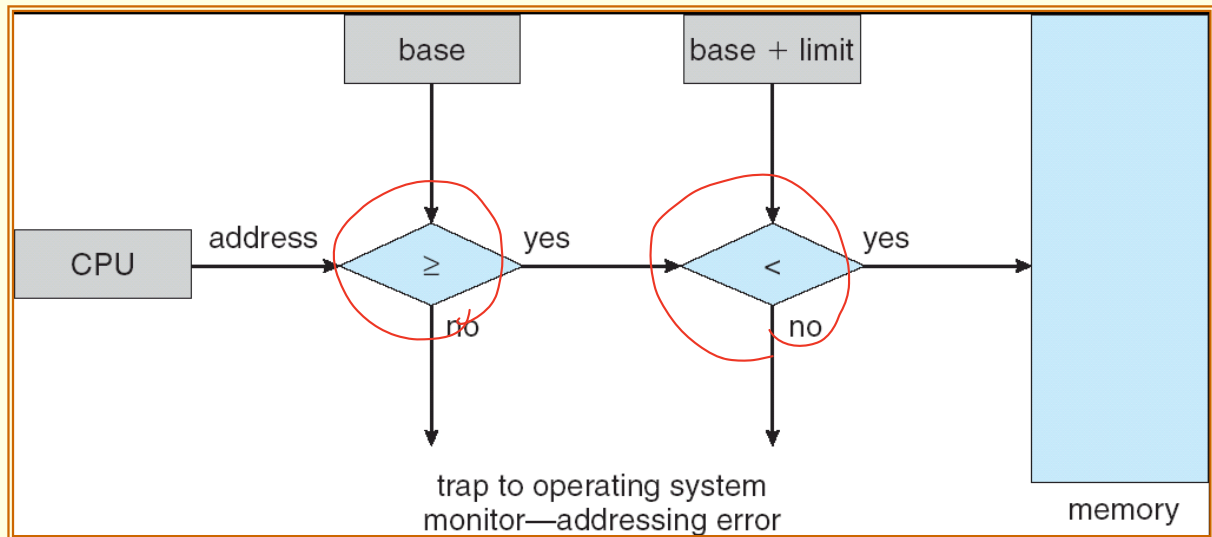
- We must ensure correct memory operations ...
 - To protect the operating system from access by user processes .
 - To protect user processes from one another.
- To do this ...
 - Each process has a range of **legal** addresses.
 - Process can access only these legal addresses.
- We can provide this protection by using two registers:
 - ① ■ **Base register** – hold the smallest legal physical memory address. ↘ 第一個門牌
 - ② ■ **Limit register** – specify the size of the range. ↘ 多大?

Background (3/5)



Background (4/5)

- Then, we compare every address generated **in user mode** with the registers.
 - Any attempt (in user mode) to access operating-system memory or other users' memory results in a fatal error.
 - Therefore, we can prevent a user program from modifying the code or data structures of either the operating system or other users.

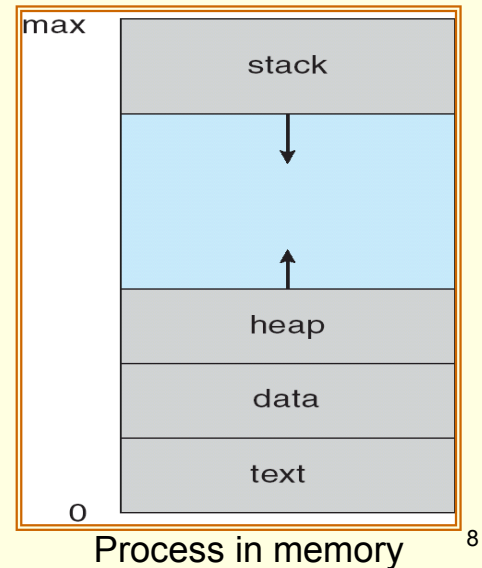


Background (5/5)

- Note that, the registers can be loaded only by the operating system.
 - To prevent user programs from changing the registers' contents.
- The operating system, executing in kernel mode, is given unrestricted access to both operating system and users' memory.
 - Allow the operating system to load users' program into users' memory, to dump out those programs in case of errors ...

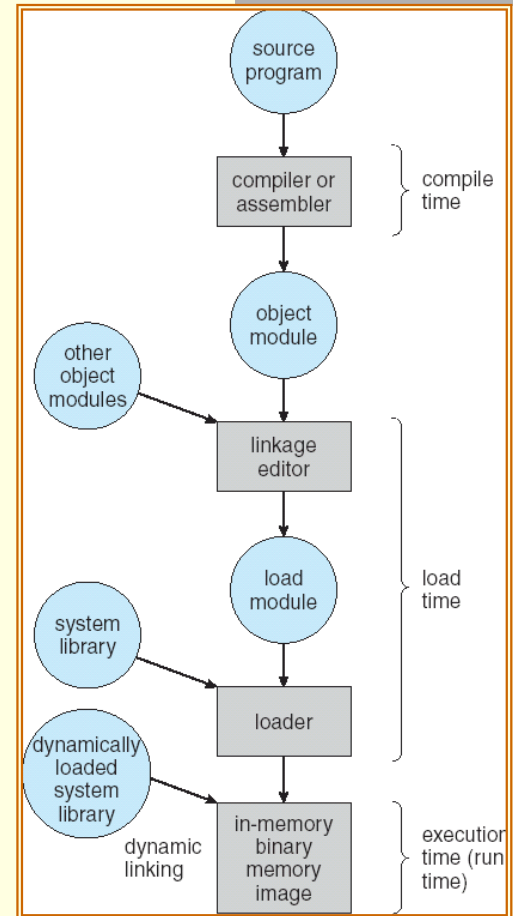
Address Binding (1/4)

- Usually, a program resides on a disk as a binary executable file.
- To be executed, the program must be brought into memory and placed within a process.
- The process may be moved between disk and memory during its execution.
 一下可能在 disk
 一下可能在 memory
- The processes on the disk that are waiting form the **input queue**.



Address Binding (2/4)

- Before being executed, a user program will go through several steps (some of which may be optional).
- Memory addresses may be represented in different ways during these steps.
- In the source program, addresses are generally **symbolic** (such as **variable count**).
人類看得懂的
- Generally, a compiler will bind these symbolic addresses to **re-locatable addresses** (such as 14 bytes from the beginning of this module).
編譯時 symbolic → relocatable
動態開頭有多遠



Address Binding (3/4)

要執行時

Typically, the loader (or linkage editor) will **bind** the relocatable addresses to absolute addresses (such as 74014).

$\text{relocatable address} + \text{process 開始的 address} = \text{absolute address}$

- **Binding** is a mapping from one address space to another.

- Classically, the binding to physical memory address can be done at any step along the way:
↓
但 bind 之後就不能換位子了

- **Compile time:**

- If you know at compile time where the process will reside in memory, then absolute code (address) can be generated.
- If, at some later time, the starting location changes, then it will be necessary to **recompile** this code.

Address Binding (4/4)

■ Load time:

- If it is not known at compile time where the process will reside in memory, then the compiler must generate **re-locatable code**.
- Binding is then delayed until **load time**.
- If the starting address changes, we need only reload the code to incorporate this changed value.

■ Execution time:

- If the process can be moved during its execution from one memory segment to another, then binding must be delayed until **run** (execution) **time**.
- Most general-purpose operating systems use this method.

Logical vs. Physical Address Space (1/2)

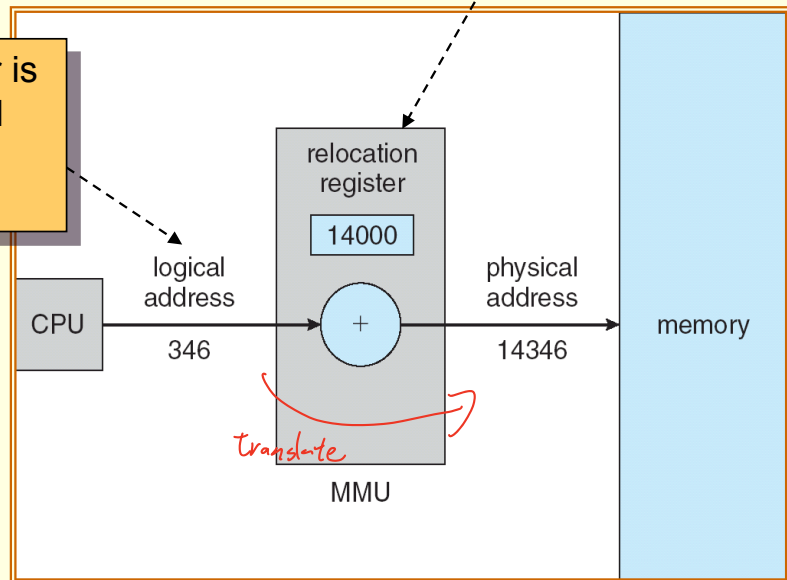
- **Logical address** - an address generated by the CPU (or a program).
 - The set of all logical addresses generated by a program is a **logical address space**. A hand-drawn diagram showing a rectangular box divided into two sections. The left section is labeled '00' and the right section is labeled '01'.
- **Physical address** – an address seen by the **memory unit**.
 - The set of all physical addresses corresponding to the logical addresses is a **physical address space**.
- For compile-time and load-time address-binding methods, logical and physical addresses are **identical**.
- However, the execution-time address-binding scheme results in **differing** logical (virtual) and physical addresses.
 - Process only runs in logical locations 0 to max .
 - Logical addresses must be mapped to physical addresses before access.

Logical vs. Physical Address Space (2/2)

- **Memory-management unit** (MMU) – a hardware device that maps from virtual to physical addresses run-time.
 - There are many different methods to accomplish such mapping.
- A simple MMU scheme:
 - A register-based scheme.

The value in the relocation register is **added** to every address generated by a user process at the time it is send to memory.

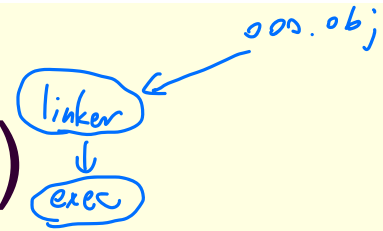
Another name of the base register.



Dynamic Loading

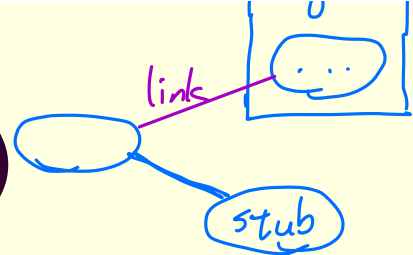
- With **dynamic loading**, a routine (of a program) is not loaded until it is called.
 - Routines are kept on disk in a **re-locatable load format**.
 - The main program is loaded into memory and is executed.
 - When we need (call) a routine, the caller first checks to see whether the routine has been loaded.
 - If not ...
 - The loader is called to load the desired routine into memory.
 - Update the program's address space to reflect this change.
 - Pass control to the newly loaded routine.
- Benefits of dynamic loading:
 - Unused routines, usually error-handling routines, are never loaded.
 - Better memory-space utilization.

Dynamic Linking (1/4)

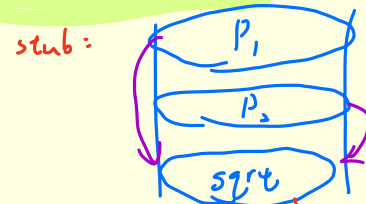


- Some operating systems support only **static linking**.
 - Compiled object modules are combined by the loader (or linker) into the binary program image.
- **Dynamic linking** – linking is postponed until execution time.
 - Usually used with system libraries.
- Without this facility ...
 - Each program (using system libraries) must include a copy of the library in the executable image.
 - Waste both disk space and main memory.

Dynamic Linking (2/4)



- A **stub** is included in the image for each library routine reference.
- When the stub is executed ...
 - It checks to see whether the needed library routine is already in memory.
 - If not, the routine is loaded into memory.
 - Either way, the stub replaces itself with the address of the routine and executes the routine.
- Under this scheme, all processes that use a library execute **only one copy of the library code!!**



Dynamic Linking (3/4)

- This scheme can be extended to **library updates** (such as bug fixes).
 - A library may be replaced by a new version.
 - All programs that reference the library will **automatically** use the new version.
- Usually version information is included in both the program and the library, so that programs will not accidentally execute new, incompatible versions of libraries.
- More than one version of a library may be loaded into memory.
 - Only programs that are compiled with new library version are affected by the changes.
 - Other programs linked before the new library was installed will continue using the older library.
 - This system is known as **shared libraries**.

不同 library 会 load 进来

Dynamic Linking (4/4)

- Dynamic linking generally requires help from the operating system.
 - The operating system is the only entity that can check to see whether the needed routine is in another process's memory space.
 - And allow multiple processes to access the same memory addresses.

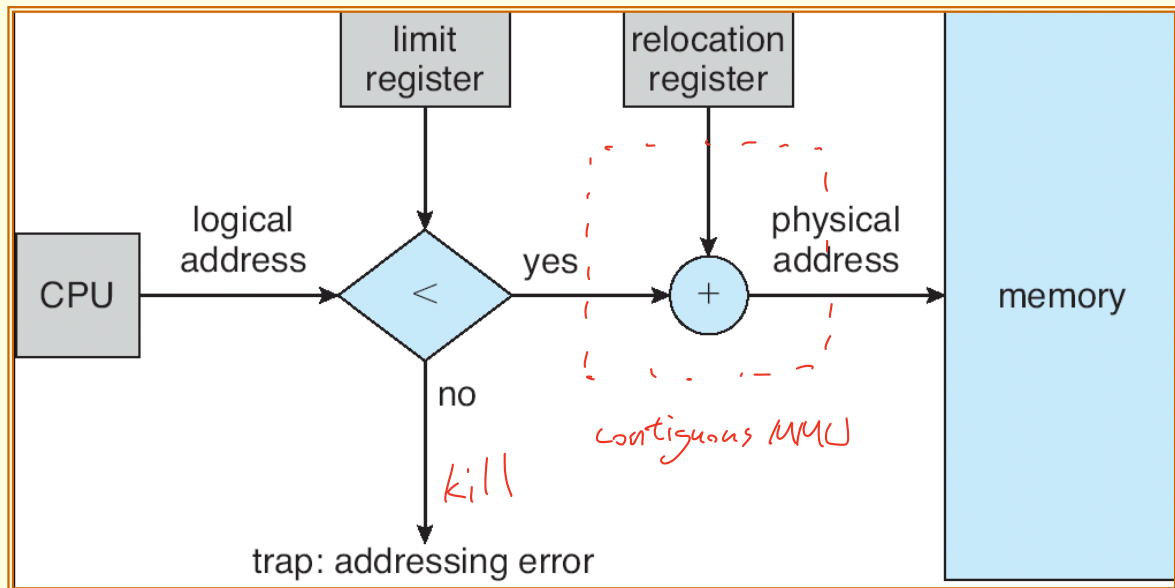
Contiguous

Contiguous Memory Allocation (1/6)

- The memory is usually divided into two partitions:
 - One for operating system.
 - The operating system can be placed in either **low** memory or **high** memory.
 - Due to the interrupt vector (which is often in low memory), operating system is usually placed in low memory.
 - One for the user processes.
 - We want several user processes to reside in memory.
 - In **contiguous memory allocation**, each process is contained in a single contiguous section of memory.

Contiguous Memory Allocation (2/6)

- Before discussing memory allocation, we talk about **memory mapping and protection**.
 - The MMU consists of a re-location register and a limit register.

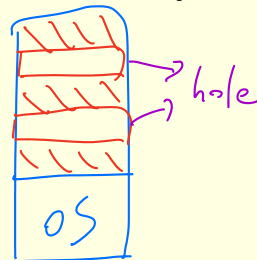


Contiguous Memory Allocation (3/6)

- One of the simplest methods for allocating memory is to divide memory into several **fixed-sized partitions**. 區間固定
 - Each partition can contain exactly one process.
 - The degree of multiprogramming is bound by the number of partitions.
- When a partition is free, a process is selected from the input queue and is loaded into the free partition.
- When the process terminates, the partition becomes available for another process.
- This method was used by IBM OS/360 operating system (called MFT).
 - **The method is out-of-date and no longer in use!!**

Contiguous Memory Allocation (4/6)

- A generalization of the fixed-partition scheme (called MVT, or **dynamic partitions**):
 - Initially, all memory is available for user processes.
 - It is considered one large block of available memory, a **hole**.
 - When a process arrives and needs memory, we (OS) search for a hole large enough for this process. 找一个足够大的洞
 - If we find one, we allocate only as much memory as is needed.
 - Keeping the rest available to satisfy future requests.



Contiguous Memory Allocation (5/6)

- At any given time, we have the input queue (list of memory-waiting processes) and a set of holes of various sizes scattered throughout memory.
- To load a waiting process ...
 - The system searches the set for a hole that is large enough for this process.
 - If the hole is too large, it is **split** into two parts.
 - One for the process, the other is returned to the set of holes.
- When a process terminates ...
 - It releases its block of memory, which is placed back in the set of holes.
 - If the new hold is adjacent to other holes, these adjacent holes are **merged** to form one larger hole.

Contiguous Memory Allocation (6/6)

- **Dynamic storage-allocation problem** – how to satisfy a request of size n from a list of free holes?
 - **First-fit**: Allocate the first hole that is big enough.
 - **Best-fit**: Allocate the smallest hole that is big enough.
 - Must search entire list, unless ordered by size.
 - Produces the smallest leftover hole. → 用掉之後剩下的空間
 - **Worst-fit**: Allocate the largest hole.
 - Must also search entire list.
 - Produces the largest leftover hole, which may be more useful than the smaller left from a best-fit approach.
- Simulations have shown that both first-fit and best-fit are better than worst-fit in terms of decreasing time and storage utilization.
- First-fit is generally faster than best-fit and has similar storage utilization.

Fragmentation (1/2)

- As processes are loaded and removed from memory, the free memory space is broken into little pieces.
- **External fragmentation:**
 - There is enough **total** memory space to satisfy a request.
 - But the available spaces are not contiguous, and fragmented into a large number of small holes.
 - The first-fit and best-fit strategies usually suffer from this fragmentation.
- **50-percent rule:** statistical analysis of **first-fit** reveals that, given N allocated blocks, another $0.5 N$ blocks will be lost to fragmentation.
 - **One-third of memory may be unusable!!**

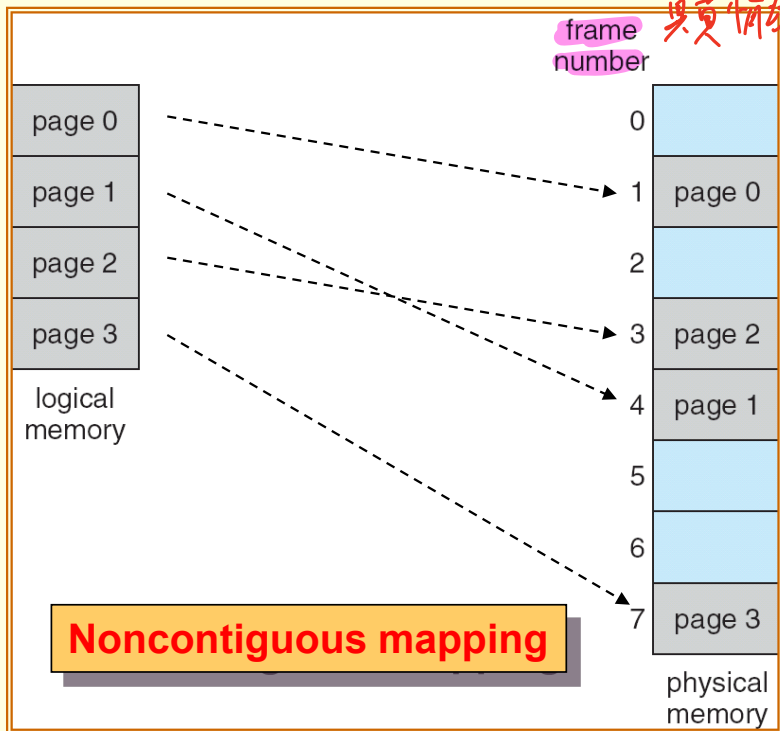
Fragmentation (2/2)

- **Compaction** - a solution to external fragmentation.
 - To place all free memory together in one large block.
 - Is possible only if binding is dynamic and is done at execution time.
 - The simplest compaction algorithm is to move all processes toward one end of memory.
 - All holes will move in the other direction.
- **Internal fragmentation:**
 - In the fixed-sized partition scheme, the memory allocated to a process (i.e., a partition) may be slightly larger than the requested memory.

Paging – Basic Method (1/9)

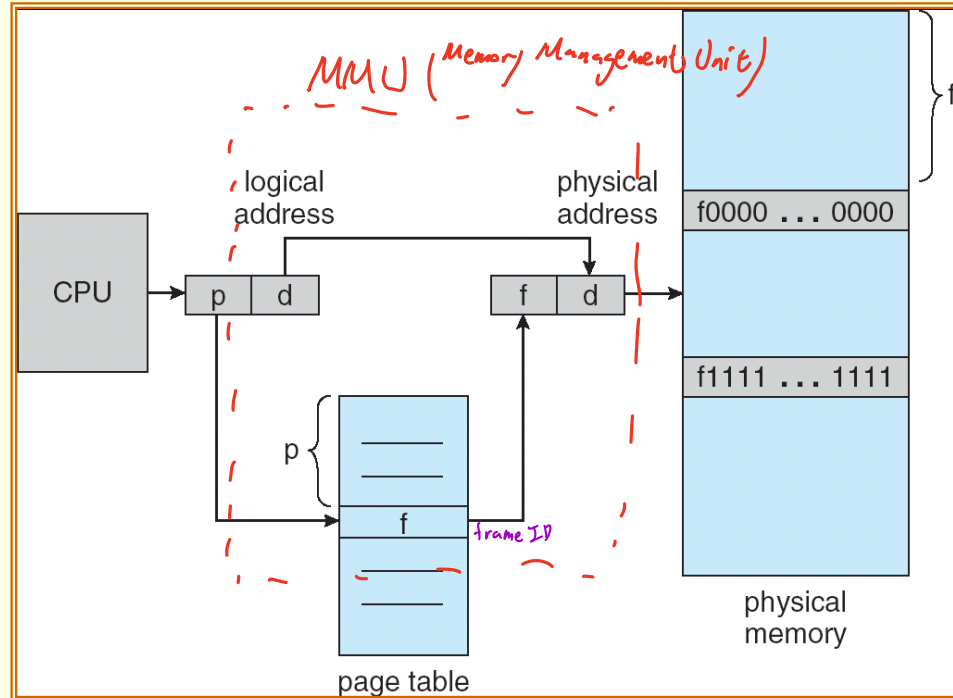
- **Paging** is a memory-management scheme that permits the physical address space of a process to be noncontiguous.
 - It is commonly used in most operating systems.
- Paging breaks ...
 - Physical memory into fixed-sized blocks called **frames**.
 - Logical memory into blocks of the same size called **pages**.

page size = frame size

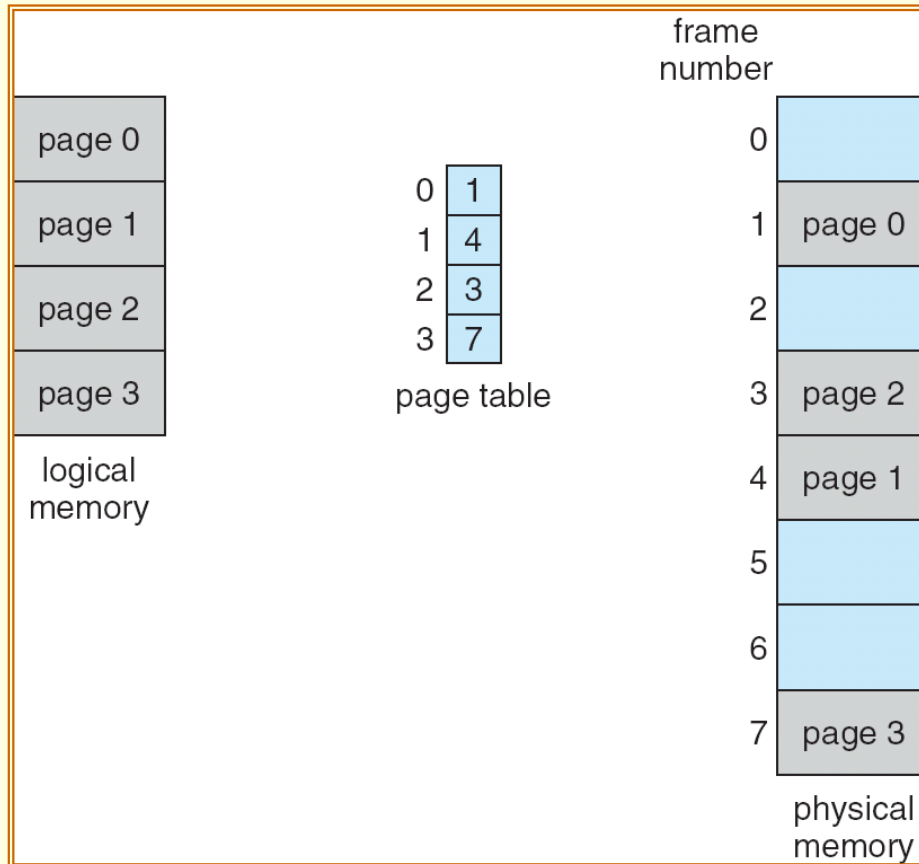


Paging – Basic Method (2/9)

- How ?? ... every logical address is divided into two parts:
 - **Page number (p).**
 - Used as an **index** into a **page table**.
 - The page table contains the base address of each page in physical memory.
 - **Page offset (d).**
 - Page offset is combined with the **base address** to **define the physical memory address**.



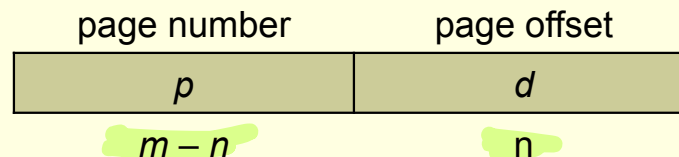
Paging – Basic Method (3/9)



Paging – Basic Method (4/9)

2 進 制

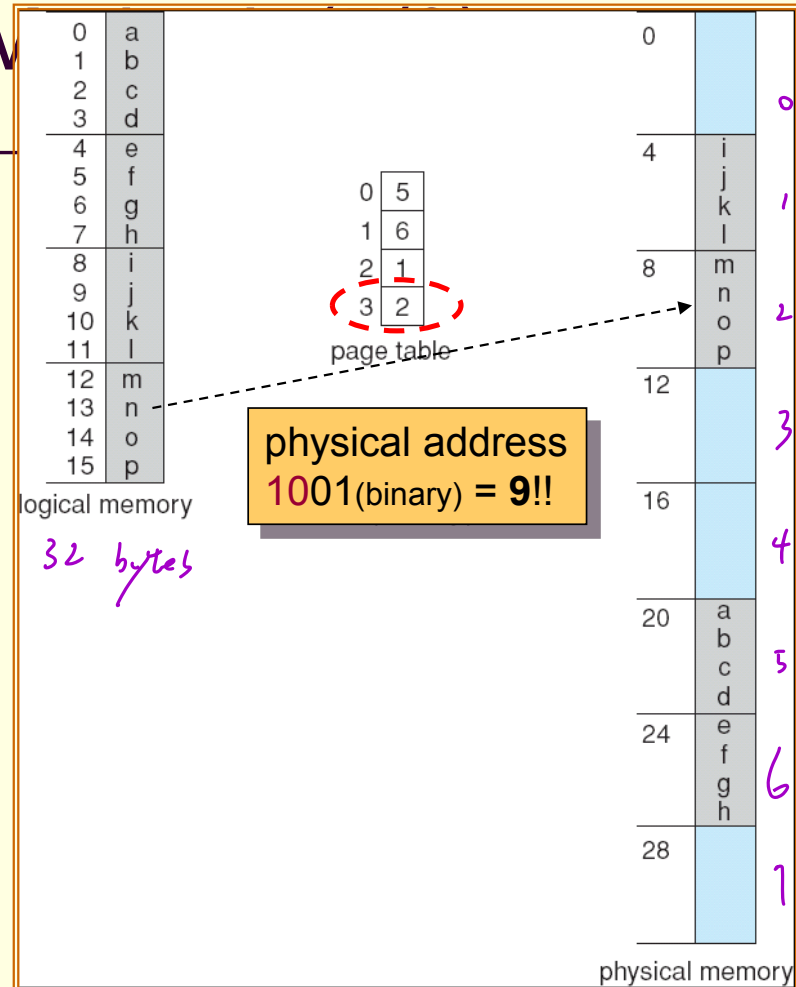
- The page size is typically a power of 2, varying between 512 bytes and 16MB per page.
 - Power of 2 makes the translation of a logical address into a page number and page offset particular easy.
- If the size of logical address space is 2^m and a page size is 2^n addressing units (e.g., bytes) ...
 - The high-order $m - n$ bits of a logical address designate the page number.
 - The n low-order bits designate the page offset.



$$\frac{2^m}{2^n}$$

Paging – Basic M

- Example:
 - Page size 4 bytes.
 - $n = 2$.
 - A physical memory of 32 bytes (8 pages).
- What is the physical address of logical address 13 ??
 - $13 \rightarrow 1101$ (binary) *← logical address*
 - Page number = 11 (binary) = 3.
 - Page offset = 01 (binary).



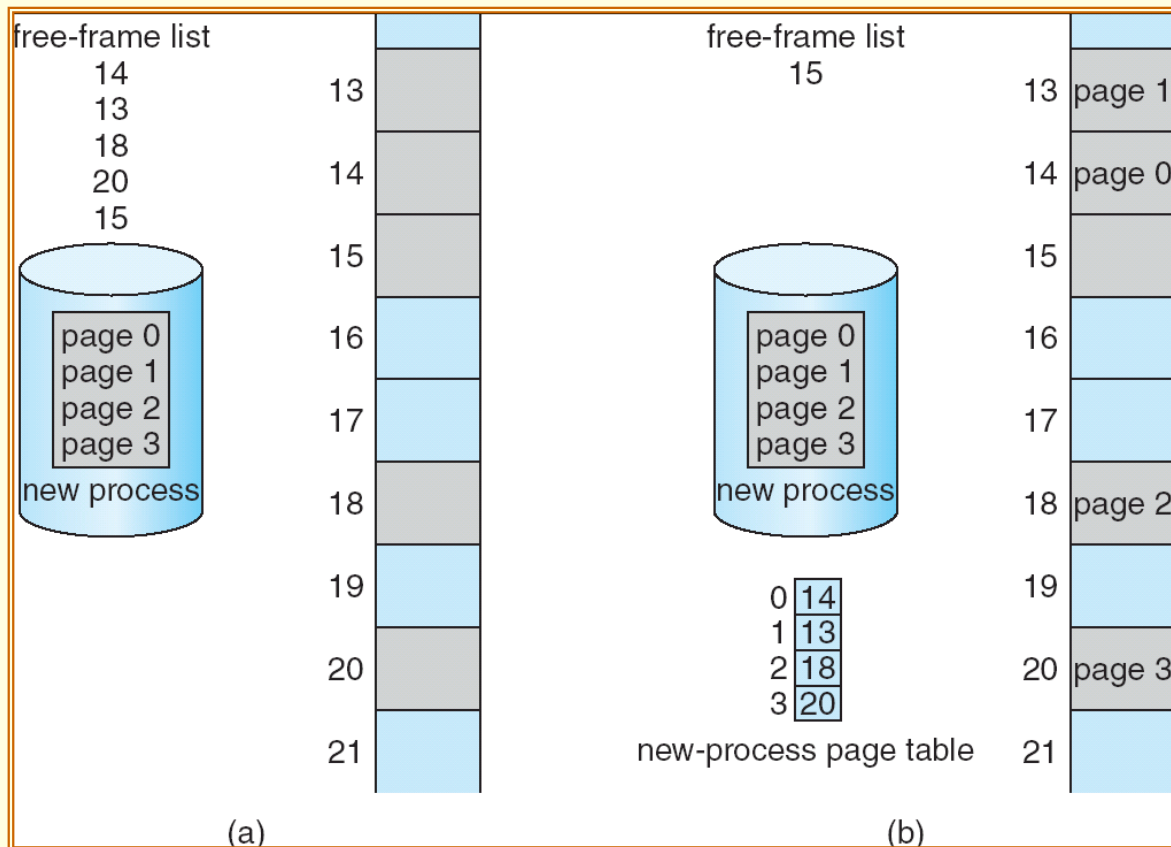
Paging – Basic Method (6/9)

- For a page table with 32-bit (4 bytes) entry length ...
 - The table can point to 2^{32} physical page frames.
 - If frame size is 4 KB (12 bits), then the system can address 2^{44} bytes (or 16 TB) of physical memory.

page table

0	A 32-bit page number
1	A 32-bit page number
⋮	

Paging – Basic Method (7/9)



Paging – Basic Method (8/9)

- Paging itself is a form of **dynamic relocation**.
 - Every logical address is bound by the paging hardware to some physical address.
- When using paging, **we have no external fragmentation!!**
 - Any free frame can be allocated to a process that needs it.
- However, **we may have some internal fragmentation**.
 - The last frame allocated may not be complete full.
 - In worst case, a process need n pages plus 1 byte. It would be allocated $n+1$ frames.
 - Resulting in an **internal fragmentation** of almost an entire frame.

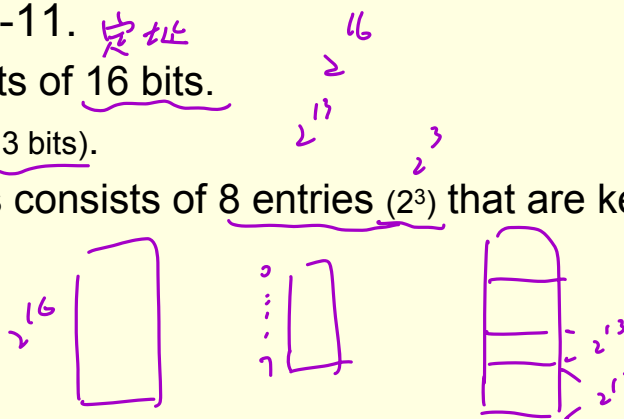


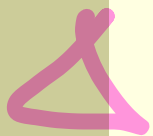
Paging – Basic Method (9/9)

- We can expect internal fragmentation to average one-half page per process.
 - This consideration suggests that small page sizes are desirable.
- However ...
 - Overhead is involved in each page-table entry. 變大
 - Also, disk I/O is more efficient when the number of data being transferred is larger.
- To know the status of physical memory, the operating system generally has a data structure call a **frame table**.
 - The frame table has one entry for each physical page frame.
 - Indicating whether the frame is free or allocated.
 - If allocated, to which page of which process.

Paging – Hardware Support (1/6)

- How to implement paging?
- In the simplest case, the page table is implemented as a set of dedicated registers.
- Most operating systems allocate a page table for each process.
 - So, the CPU dispatcher reloads these registers during context switching.
- Example – DEC PDP-11. *定址*
 - The address consists of 16 bits.
 - Page size is 8 KB (13 bits).
 - The page table thus consists of 8 entries (2^3) that are kept in registers.



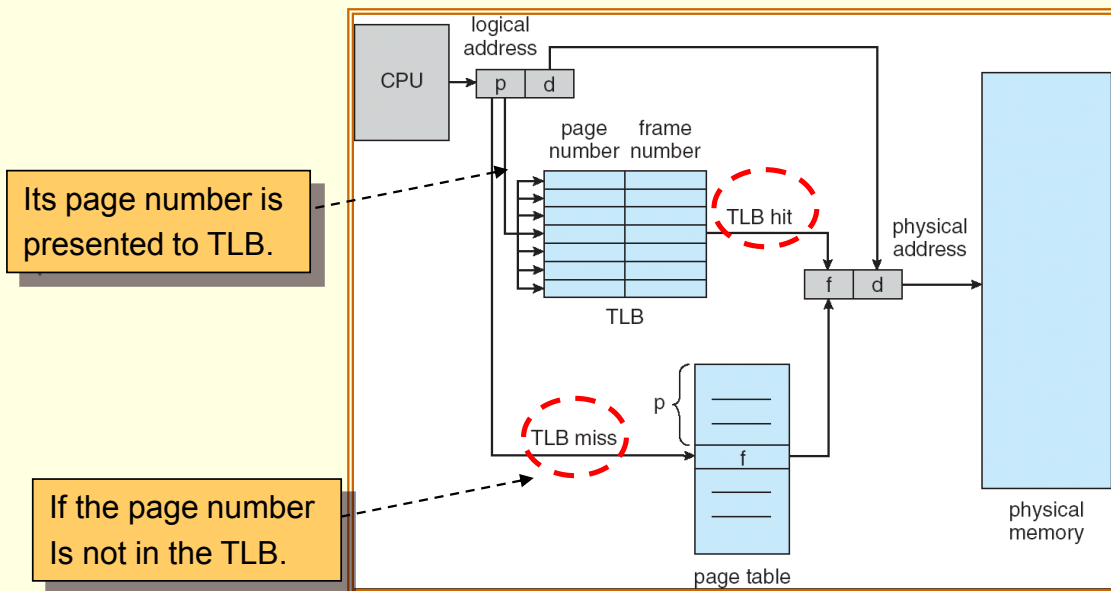


Paging – Hardware Support (2/6)

- The use of fast registers is not feasible!!
 - Most contemporary computers allow the page table to be very large.
- Rather, the page table is kept in main memory, and a **page-table base register** (PTBR) points to the page table.
 - Problem – **two** memory accesses are needed!!
 - If we want to access location i , we must first index into the page table and combine the frame address with the page offset to produce the actual address.
- A solution to this problem is to use a special **hardware cache**, called a **translation look-aside buffer** (TLB).

Paging – Hardware Support (3/6)

- Each entry in the TLB consists of two parts: a key (page number) and a value (frame number).
 - Typically, the number of entries in a TLB is small (64 to 1024), because the hardware is expensive.
- When a logical address is generated ...



Paging – Hardware Support (4/6)

- We will add the page number and frame number of a TLB miss to the TLB.
 - They will be found quickly on the next reference.
 - If the TLB is full, the operating system must select one for replacement.
 - Replacement policies range from **least recently used** (LRU) to random (chapter 9 for more details).
- **Hit ratio** – the percentage of times that a page number is found in TLB.
 - If it takes 20 ns to search the ^{TLB}~~TLB~~, and 100 ns to access memory.
 - A TLB hit takes 120 ns to access physical memory.
 - If we fail to find the page number in the TLB (20 ns), then we must first access memory for page table and frame number (100 ns) and then access the desired byte in memory (100 ns).
 - A total of 220 ns

Miss

Paging – Hardware Support (5/6)

- To find the **effective memory-access time**, we weight each case by its probability.
 - For a 80-percent hit ratio:

$$\begin{aligned}\text{effective access time} &= 0.80 \overset{\text{hit}}{\times 120} + 0.20 \overset{\text{miss}}{\times 220} \\ &= 140 \text{ ns.}\end{aligned}$$

Paging – Hardware Support (6/6)

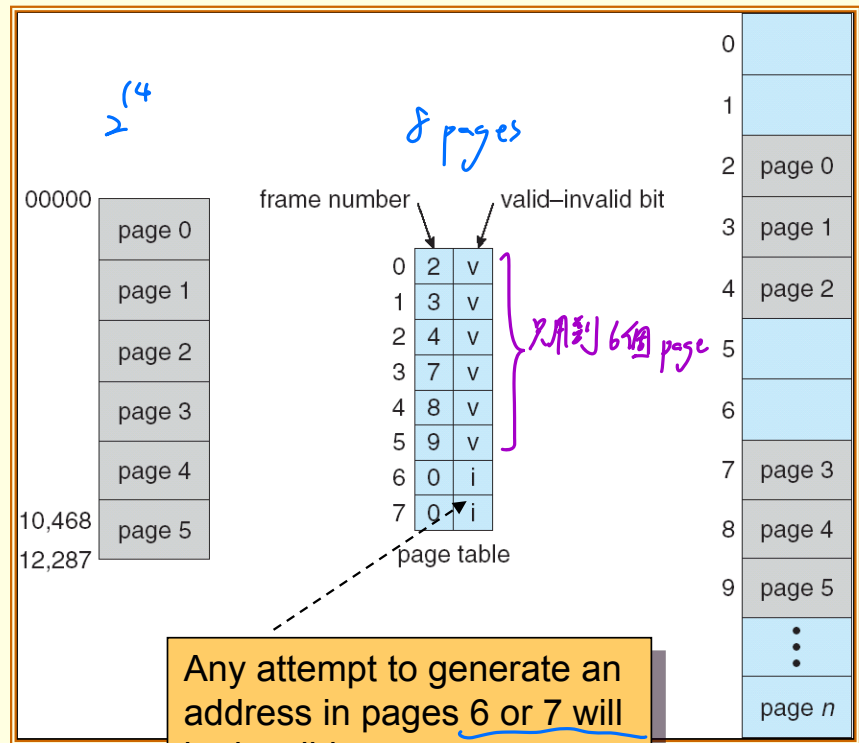
- The TLB contains entries for several different processes simultaneously.
- To ensure address-space protection ...
 - Some TLBs store **address-space identifier** (ASIDs) in each TLB entry.
 - The ASID for the currently running process must match the ASID associated with the virtual page.

Paging – **Protection** (1/3)

- We can provide separate protection bits for each page.
 - When the physical address is being computed, the protection bits can be checked.
 - Bits define *read-only*, *read-write*, *execute-only*, ...
- One general bit is valid-invalid bit.
 - When the bit is set to invalid, the page is not in the process's logical address space.

Paging – Protection (2/3)

- Suppose, a system with 14-bit (logical) address space.
- Page size is 2 KB (11 bits).
- Page table has $2^{(14-11)} = 8$ pages.
- A program uses addresses 0 to 10468.
 - Require $10468 / 2^{11} = 5.11 \rightarrow 6$ frames.
- The internal fragmentation problem.
 - Not all references to page 5 are valid!!



invalid 就不幫翻譯成 physical address
直接 kill

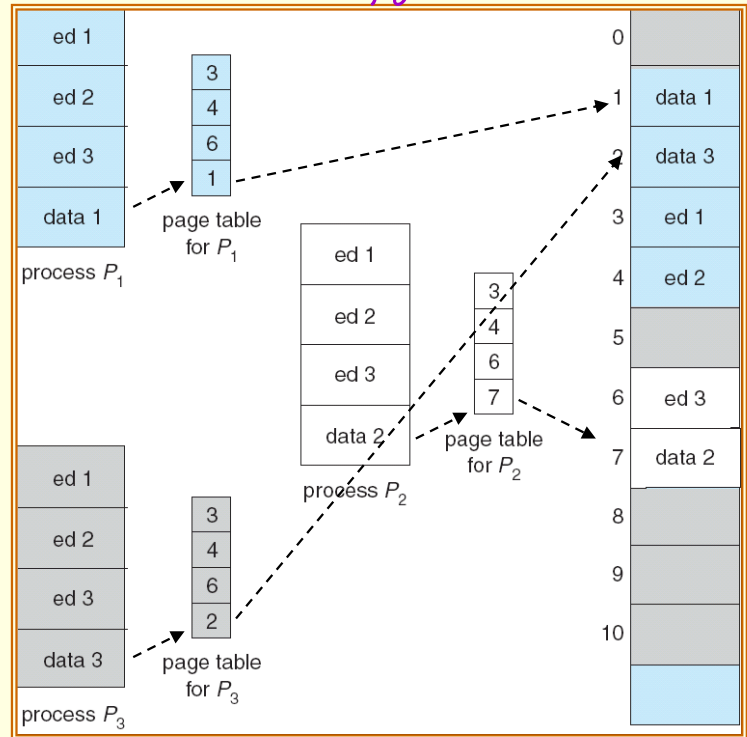
Paging – Protection (3/3)

- Rarely does a process use all its (logical) address range.
 - Previous example: 32-bit entry length with 4 KB frame size results in **16 TB** of physical memory.
- It would be wasteful to create a page table with entries for every page in the address range.
- Some systems provide hardware, in the form of a **page-table length register** (PTLR), to indicate the size of the page table.
 - The value is checked against every logical address to verify that the address is in the valid range.

Paging – Shared Pages (1/2)

- Consider a system where 40 users execute a text editor.
 - If the text editor consists of 150 KB of code and 50 KB of data space, we need 8,000 KB to support the 40 users.
- Paging makes the common code sharing easier.
 - If the code is reentrant (pure) code, **its pages can be shared**.

每个 process 都有自己的 page table



Paging – Shared Pages (2/2)

- Each user's page table maps onto the same physical copy of the editor, but data pages are mapped onto different frames.
 - To support 40 users, the total space required is 2150 KB – a significant savings.
- In addition to code sharing ...
 - In Chapter 4, we discussed the sharing of the address space of a task by thread.
 - In Chapter 3, we described shared memory.
 - Some operating systems implement the sharing using shared pages.

Hierarchical Paging (1/4) 前補充講義

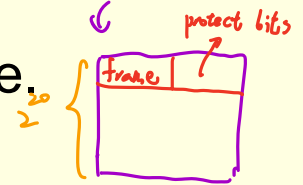
- Most systems support a large logical address space (2^{32} to 2^{64}).

- The page table itself becomes excessively large.



- A system with a 32-bit logical address space.

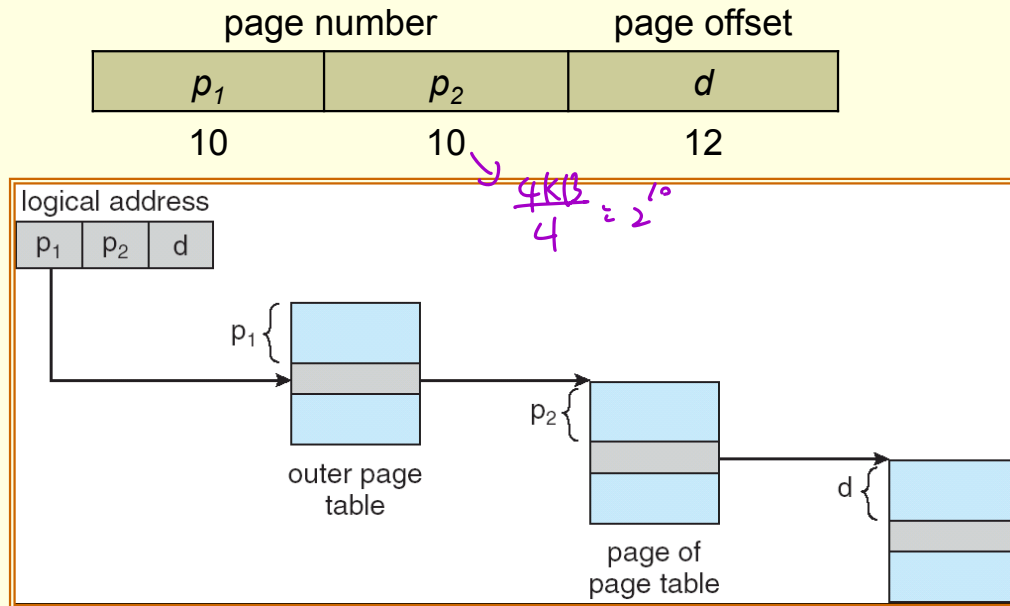
- The page size is 4 KB (2^{12}).
 - Then, a page table consists of 2^{32-12} (million) entries.
 - If each entry consists of 4 bytes, each process may need up to **4 MB** of physical address for the page table.



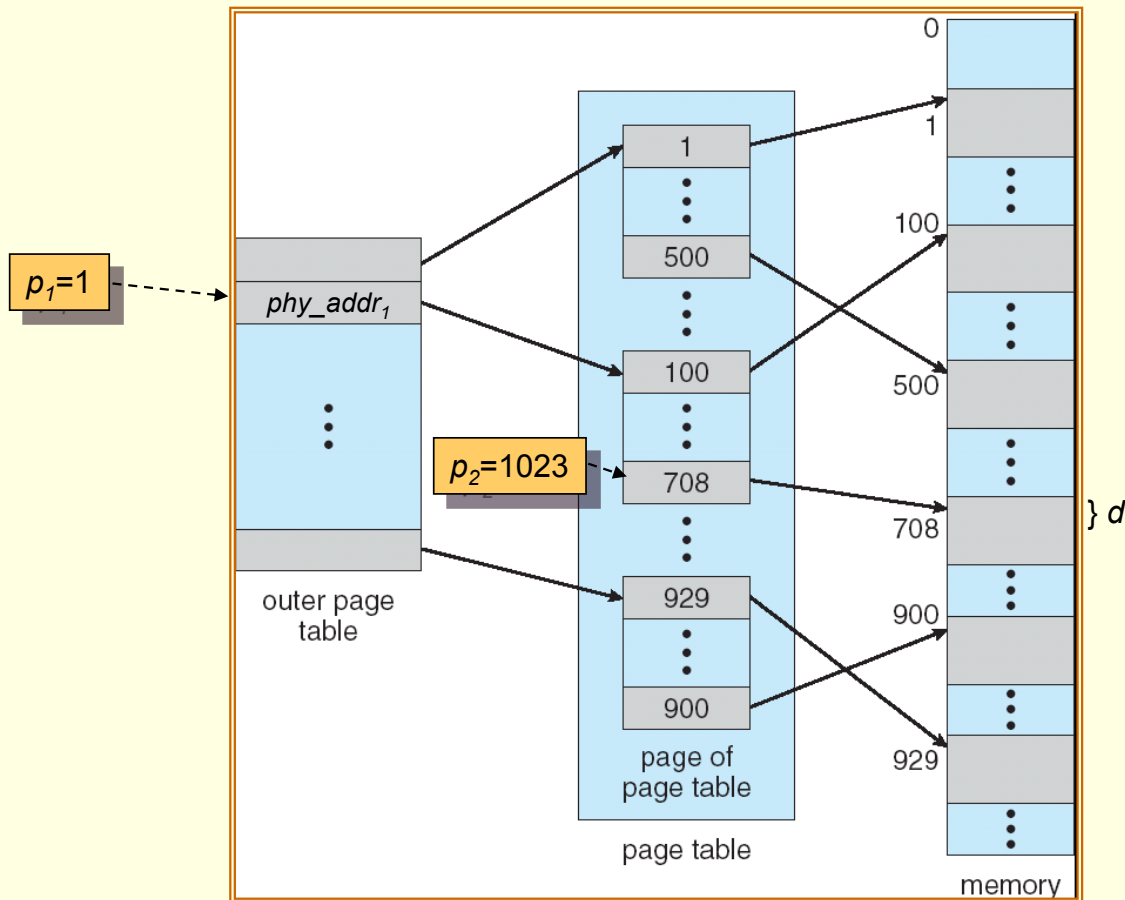
- It is inappropriate to allocate the large page table **contiguously** in main memory.

Hierarchical Paging (2/4)

- **Two-level paging** (also known as a **forward-mapped page table**)
 - **The page table itself is also paged.**
 - we can further divide the *page number* into two parts:
 - p_1 is an **index** into the outer page table.
 - p_2 is the **displacement** within the page of the outer page table.



Hierarchical Paging (3/4)

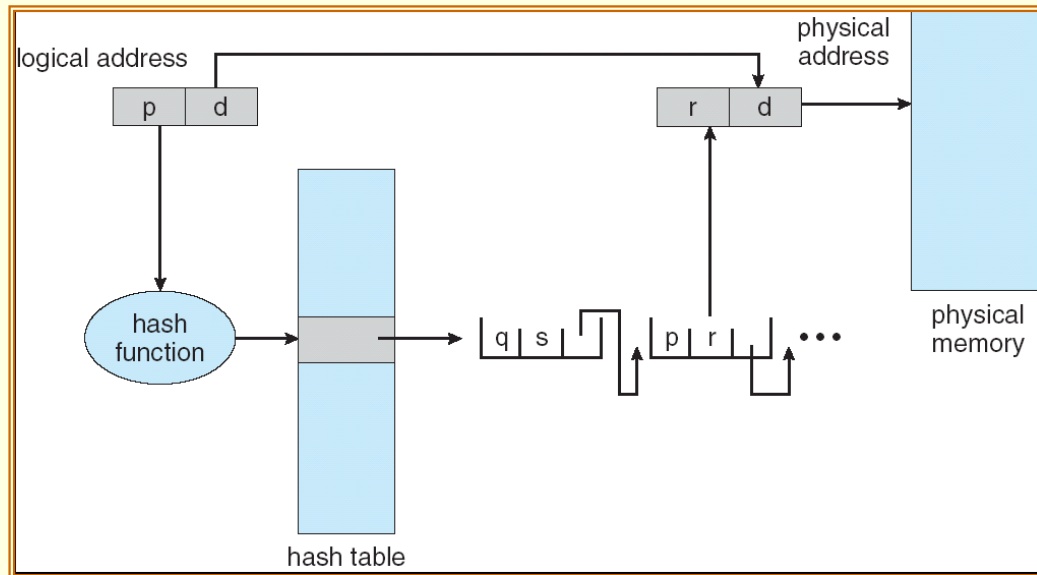


Hierarchical Paging (4/4)

- For a system with a **64-bit** logical-address space, a two-level paging scheme is not appropriate!!
 - Suppose that the page size is 4 KB (2^{12}).
 - Let the inner page tables be one page long (or contain 2^{10} 4-byte entries).
 - The outer page table consists of $2^{(64-12-10)}$ 4-byte entries, or **2^{44} bytes**.
 - We can page the outer page table, giving us a three-level paging scheme.
 - But the out page table is still **2^{34} bytes** in size.
- Hierarchical page tables are generally inappropriate for 64-bit architecture.
 - For example, the 64-bit UltraSPARC would require **seven levels** of paging.
 - Result in a prohibitive number of memory access to translate each logical address.

Hashed Page Tables

- A common approach for handling address spaces larger than 32 bits.
- The **virtual page number is hashed** into the hash table.
- Each entry in the has table contains a **linked list** of elements that has to the same location (to handle collisions).

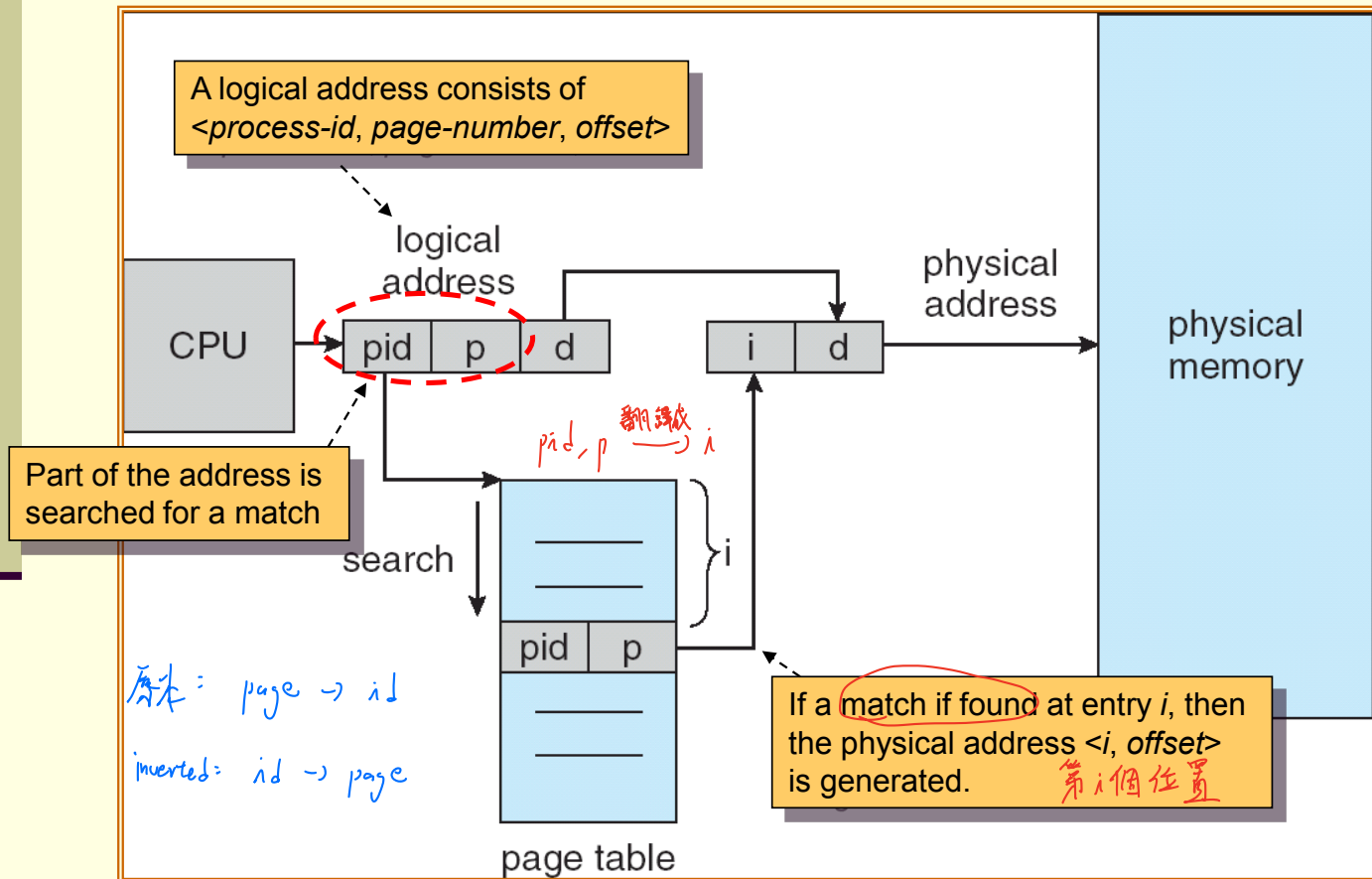


可以省記憶體

Inverted Page Tables (1/3)

- Usually, each process has an associated page table.
 - The table has one entry for each page that the process is using (or one slot for each page, regardless of validity).
 - Each page table may consist of millions of entries!!
 - So ... the tables of all processes may consume large amount of physical memory.
- The scheme of *inverted page table* uses **only one page table** in the system.
 - The table has one entry for each page of physical memory.
 - Each entry consists of:
 - The process that owns that page.
 - The virtual address of the page stored in that real memory location.

Inverted Page Tables (2/3)



Inverted Page Tables (3/3)

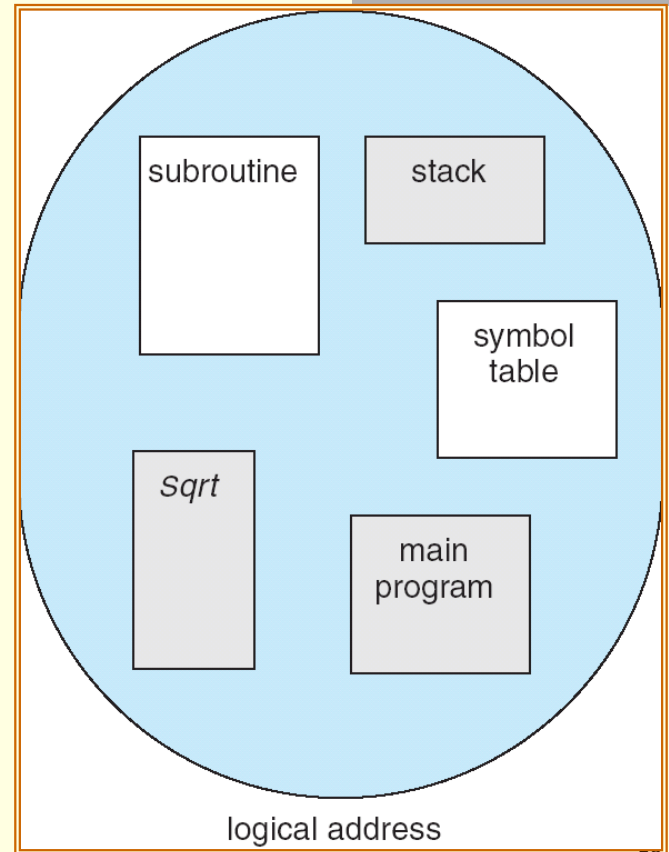
- Although this scheme **decreases** the amount of memory requirements ...
 - It **increases** the amount of time needed to search the table.
 - The whole table might need to be searched for a match.
- The scheme also has difficulty implementing shared memory.
 - There is **only one** virtual page entry for every physical page.

Segmentation (1/5)

- Do users think of memory as a linear array of bytes??
 - No ...
- We think of a program as:
 - A main program,
 - A set of methods, procedure, or functions.
 - Various data structures: objects, arrays, stacks.
- Each of these modules or data elements is **referred to by name**.
 - You talk about them (the stack, the main program, ...) without caring what addresses in memory these elements occupy.

Segmentation (2/5)

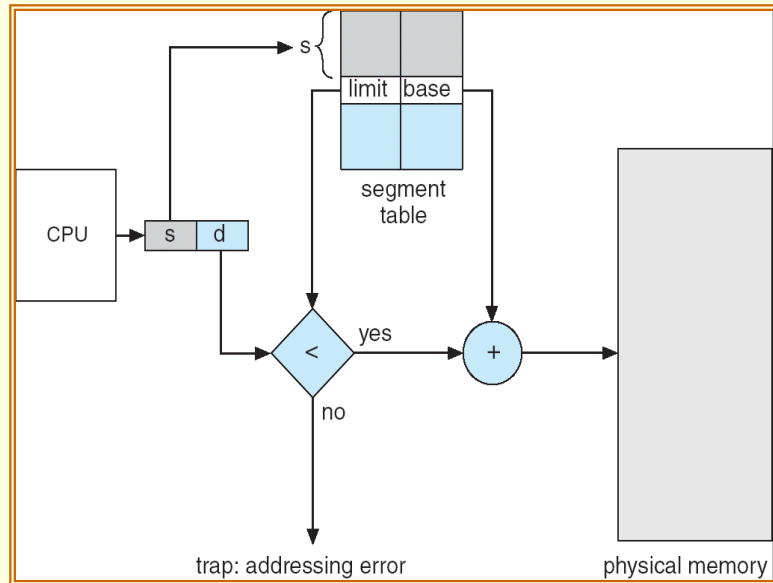
- Segmentation supports this user view of memory.
- A logical address space is a collection of segments.
 - Each segment has a name and a length.
 - The (logical) addresses specify both the segment name (usually a number) and the offset within the segment.
 - A two-tuple representation of a logical address:
<segment-number, offset>



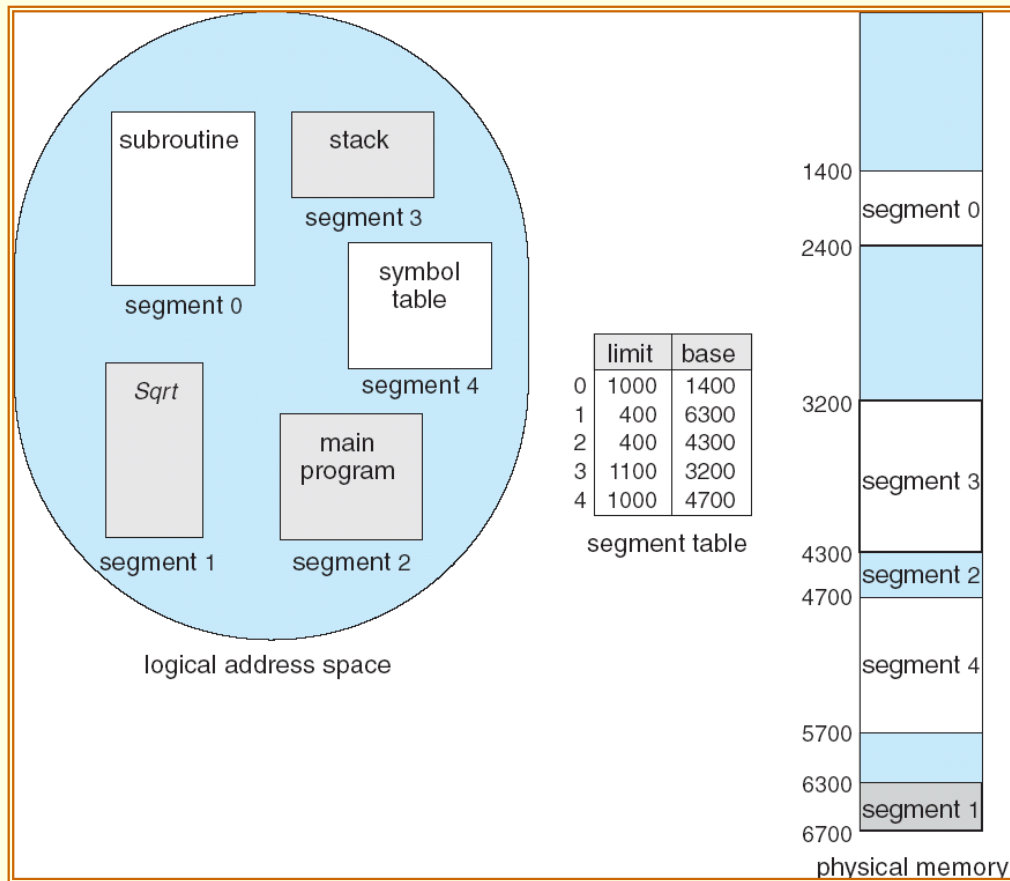
Segmentation (3/5)

- In this memory-management scheme, a C compiler might create separate segments for the following:
 - The code.
 - Global variables.
 - The heap, from which memory is allocated.
 - The stacks used by each thread.
 - The standard C library.

- Each entry has:
 - a segment base – the starting physical address of the segment.
 - a segment limit – the length of the segment.
- The table is thus essentially an **array** of base-limit **register pairs**.



Segmentation (5/5)





End of Chapter 8

